IAI600 – Introduction to Applied Artificial Intelligence

Lab V: Unsupervised Learning

Main Topic: Unsupervised Learning and K-Means Clustering
Dataset: Iris dataset and synthetic datasets
Software: Python, Jupyter Notebook / Google Colab, Scikit-learn
Disclaimer: This lab material has been prepared with the assistance of Generative AI and is based on Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow; please report any errors, inaccuracies, or limitations you identify.

Objective: The objective of this lab is to understand how unsupervised learning can be used to discover structure in data without predefined target labels. You will work mainly with K-Means clustering and investigate cluster assignments, centroids, decision boundaries, model evaluation, initialization, Mini-Batch K-Means, and methods for selecting the number of clusters.

Part I – Loading and Exploring the Iris Dataset

The Iris dataset contains measurements of iris flowers. The dataset includes four numerical features: Sepal length, Sepal width, Petal length, Petal width.
The original dataset also contains species labels. However, in this lab we will initially ignore these labels and treat the problem as an unsupervised learning problem.
Import the Required Libraries: Run the following code:

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.cluster import MiniBatchKMeans
```

Explain the purpose of each imported library or class.
Load the Dataset

```
iris = load_iris()

X = iris.data
y = iris.target

print("Shape of X:", X.shape)
print("Shape of y:", y.shape)
print("Feature names:", iris.feature_names)
```

Notice that both feature data and target labels are available. For the clustering experiment, however, use only:

```
X
```

and do not use:

```
y
```

for training the clustering model.

Task 1.1 – Interpretation

Answer:

1. How many observations are in the dataset?
2. How many features does each observation contain?
3. Why should the target variable not be used when performing unsupervised clustering?
4. What would happen conceptually if we used the known species labels to train a model?

Part II – Creating Data for K-Means Clustering

Before applying K-Means to the Iris dataset, we will first work with a synthetic dataset where the underlying groups are easier to visualize.
Create Synthetic Clusters: Use Scikit-learn to generate artificial data containing several groups.

```
from sklearn.datasets import make_blobs

X_blobs, y_blobs = make_blobs(
    n_samples=300,
    centers=3,
    cluster_std=1.0,
    random_state=42
)

plt.figure(figsize=(7,5))
plt.scatter(X_blobs[:, 0], X_blobs[:, 1], s=20)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Synthetic Data")
plt.show()
```

Task 2.1 –Interpretation

Inspect the resulting figure. Answer:

1. How many groups appear to exist in the data?
2. What visual characteristics make the groups distinguishable?
3. Why is this dataset easier to visualize than the original Iris dataset?
4. What does the parameter `cluster_std` control?

Part III – K-Means Clustering

K-Means attempts to divide observations into a predefined number of clusters. For a selected value of k, the algorithm repeatedly performs two main operations:

1. Assign each observation to the nearest cluster center.
2. Update each cluster center based on the observations assigned to it.

These operations are repeated until the solution converges.
Train a K-Means Model: Create a K-Means model with three clusters.

```
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)

kmeans.fit(X_blobs)
```

Inspect the learned cluster centers:

```
print("Cluster centers:")
print(kmeans.cluster_centers_)
```

Predict Cluster Assignments: Use the trained model to assign each observation to a cluster.

```
cluster_labels = kmeans.predict(X_blobs)

print(cluster_labels[:20])
```

You can also obtain the assignments using:

```
cluster_labels = kmeans.labels_
```

Task 3.1 –Interpretation

Answer:

1. What do the numbers 0, 1, and 2 represent?
2. Do these numbers have any inherent meaning?
3. Why should cluster number 0 not automatically be interpreted as “the first class”?
4. What information is represented by `cluster_centers_`?

Part IV – Visualizing Clusters and Decision Boundaries

Visualize the Clusters: Plot the observations and the learned cluster centers.

```
plt.figure(figsize=(8,6))

plt.scatter(
    X_blobs[:, 0],
    X_blobs[:, 1],
    c=cluster_labels,
    s=25
)

plt.scatter(
    kmeans.cluster_centers_[:, 0],
    kmeans.cluster_centers_[:, 1],
    marker="X",
    s=200,
    edgecolor="black"
)

```

```
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("K-Means Clustering")
plt.show()
```

Task 4.1 – Interpretation

Study the plot carefully.

1. Where are the cluster centers located?
2. Which observations are assigned to each cluster?
3. Are all observations equally close to their cluster center?
4. Are there observations that appear close to the boundary between clusters?

Plot the Decision Boundaries: K-Means assigns an observation to the nearest centroid. Therefore, regions of the feature space can be associated with different centroids. Create a grid covering the feature space:

```
x_min, x_max = X_blobs[:, 0].min() - 1, X_blobs[:, 0].max() + 1
y_min, y_max = X_blobs[:, 1].min() - 1, X_blobs[:, 1].max() + 1

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 500),
    np.linspace(y_min, y_max, 500)
)

grid = np.c_[xx.ravel(), yy.ravel()]
Z = kmeans.predict(grid)

Z = Z.reshape(xx.shape)
```

Plot the resulting regions:

```
plt.figure(figsize=(8,6))

plt.contourf(
    xx,
    yy,
    Z,
    alpha=0.2
)

plt.scatter(
    X_blobs[:, 0],
    X_blobs[:, 1],
    c=cluster_labels,
    s=20
)

plt.scatter(
    kmeans.cluster_centers_[:, 0],
    kmeans.cluster_centers_[:, 1],
    marker="X",
    s=200,
    edgecolor="black"
```

```
)

plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("K-Means Decision Boundaries")
plt.show()
```

Task 4.2 –Interpretation

Answer:

1. What does each region in the plot represent?
2. How are the boundaries related to the cluster centers?
3. Why are the boundaries determined by distance to the centroids?
4. What happens to the boundaries if the centroids move?

Part V – Evaluating K-Means

Unlike supervised learning, clustering does not normally have a known target value against which predictions can simply be compared. One important measure used by K-Means is **inertia**. The inertia is related to the sum of squared distances between observations and their assigned cluster centers:

$$\text{Inertia} = \sum_{i=1}^{m} \left\| x^{(i)} - \mu_{c^{(i)}} \right\|^2$$

where $\mu_{c^{(i)}}$ is the centroid of the cluster assigned to observation i.

Calculate the Inertia: Use:

```
print("Inertia:", kmeans.inertia_)
```

Task 5.1 –Interpretation

Answer:

1. What does the inertia measure?
2. Does a smaller inertia generally indicate more compact clusters?
3. Why can inertia not be used by itself to determine the best number of clusters?

Part VI – Random Initialization and Stability

K-Means requires initial cluster centers. Different initial positions can sometimes result in different final solutions.

Run K-Means with Different Initializations: Train several K-Means models using different random seeds.

```python
for seed in [0, 1, 2, 3, 4]:

    model = KMeans(
        n_clusters=3,
        random_state=seed,
        n_init=1
    )

    model.fit(X_blobs)

    print(
        "Random state:",
        seed,
        "Inertia:",
        model.inertia_
    )
```

Task 6.1 –Interpretation

Compare the resulting inertia values.

1. Are the results identical?
2. Why can different initializations produce different results?
3. Which solution would generally be preferred if two solutions have different inertia values?
4. Why is reproducibility important when experimenting with machine learning algorithms?

Multiple Initializations with `n_init`: K-Means can automatically try several initializations and retain the best solution.
Run:

```python
kmeans_multi = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)

kmeans_multi.fit(X_blobs)

print("Inertia:", kmeans_multi.inertia_)
```

Compare this with a model using:

```
n_init=1
```

Task 6.2–Interpretation

Explain:

1. What is the purpose of `n_init`?
2. Why can multiple initializations improve the result?
3. What is the trade-off of increasing the number of initializations?

Part VII – Mini-Batch K-Means

Standard K-Means repeatedly processes the dataset when updating the cluster centers. For very large datasets, this can become computationally expensive. Mini-Batch K-Means uses smaller subsets, or mini-batches, of the data during training.

Train Mini-Batch K-Means

```
mini_batch_kmeans = MiniBatchKMeans(
    n_clusters=3,
    random_state=42,
    batch_size=50,
    n_init=10
)

mini_batch_kmeans.fit(X_blobs)

print(
    "Mini-Batch inertia:",
    mini_batch_kmeans.inertia_
)
```

Compare the standard K-Means model and Mini-Batch K-Means.

```
print("Standard K-Means:")
print("Inertia:", kmeans_multi.inertia_)

print("\nMini-Batch K-Means:")
print("Inertia:", mini_batch_kmeans.inertia_)
```

Task 7.1 –Interpretation

Answer:

1. Are the inertia values exactly the same?
2. Why might the results differ?
3. Why could Mini-Batch K-Means be useful for very large datasets?
4. What is the potential trade-off between computational speed and clustering quality?

Part VIII – Finding the Optimal Number of Clusters

The number of clusters, k, is an important parameter of K-Means. In many practical applications, the correct number of clusters is not known in advance. One common approach is to calculate the inertia for several values of k and examine the resulting curve.

Calculate Inertia for Different Values of k

```
inertias = []

for k in range(1, 10):

    model = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )
```

```
10
11      model.fit(X_blobs)
12
13      inertias.append(model.inertia_)
```

Plot the results:

```
1  plt.figure(figsize=(8,5))
2
3  plt.plot(
4      range(1, 10),
5      inertias,
6      marker="o"
7  )
8
9  plt.xlabel("Number of clusters (k)")
10 plt.ylabel("Inertia")
11 plt.title("Inertia vs. Number of Clusters")
12 plt.xticks(range(1, 10))
13 plt.grid(True)
14 plt.show()
```

Identify the Elbow: Inspect the graph and identify the point where increasing the number of clusters begins to provide substantially smaller improvements. This point is commonly referred to as the **elbow**.

Task 8.1 – Interpretation

Answer:

1. What happens to inertia when k increases?
2. Why does inertia generally decrease as more clusters are introduced?
3. Where would you place the elbow in this experiment?
4. Why would selecting the largest possible k not necessarily be useful?
5. Why is the elbow method a heuristic rather than a guaranteed solution?

Part IX – Applying K-Means to the Iris Dataset

We now return to the Iris dataset. Remember that the clustering algorithm does not use the known species labels.

Apply K-Means to Iris: Train a K-Means model using three clusters:

```
1  iris_kmeans = KMeans(
2      n_clusters=3,
3      random_state=42,
4      n_init=10
5  )
6
7  iris_kmeans.fit(X)
8
9  iris_clusters = iris_kmeans.labels_
10
11 print("Cluster centers:")
12 print(iris_kmeans.cluster_centers_)
```

```
13
14 print("\nInertia:")
15 print(iris_kmeans.inertia_)
```

Visualize Two Features: For visualization, select two features.

```
1  plt.figure(figsize=(8,6))
2
3  plt.scatter(
4      X[:, 2],
5      X[:, 3],
6      c=iris_clusters,
7      s=30
8  )
9
10 plt.xlabel("Petal length")
11 plt.ylabel("Petal width")
12 plt.title("K-Means Clustering of Iris Data")
13 plt.show()
```

Task 9.1 – Interpretation

Answer:

1. Do the observations appear to form distinct groups?
2. Which features appear to provide useful separation?
3. Are the clusters perfectly separated?
4. Why might some observations be difficult to assign to a single cluster?

Compare with the Known Labels: Only after completing the unsupervised clustering, inspect the original labels.

```
1 print("Original species:")
2 print(iris.target_names)
3
4 print("\nFirst 20 known labels:")
5 print(y[:20])
6
7 print("\nFirst 20 cluster assignments:")
8 print(iris_clusters[:20])
```

Important: The numbers assigned by K-Means do not necessarily correspond to the original species numbers. For example, K-Means cluster "0" does not automatically mean Iris species "0". The cluster labels are simply identifiers generated by the algorithm.

Task 9.2 – Interpretation

Discuss:

1. How well do the discovered clusters correspond visually to the known species?
2. Why should we not directly compare cluster number 0 with species label 0?
3. What does this experiment tell us about the difference between clustering and classification?